

Printing Prevention

Workspace ONE® data leakage prevention includes a policy for whether printing is allowed. If printing isn't allowed then end users should be blocked from sending enterprise data to print devices.

Printing prevention is configured in the Workspace ONE Unified Endpoint Manager (UEM) console. The Workspace ONE mobile Software Development Kit (SDK) facilitates prevention by the mobile application at run time. Printing prevention is available in the SDK for Android.

Table of Contents

Description.....2

Integration.....3

 Compatibility.....3

 Console Configuration.....4

 Programming Interface for Kotlin.....5

 Replacement Kotlin Code.....5

 User Interface for Android.....6

 Next Steps.....6

Appendix: Console User Interface Screen Capture.....7

Document Information.....8

Description

Workspace ONE printing prevention works as follows.

- In the UEM, in the security policies, Data Loss Prevention (DLP) is activated. In the DLP options, printing is enabled or disabled.

See [Console Configuration](#).

- The printing setting is retrieved by the SDK instance in a mobile application, at enrollment time or subsequently.
- The mobile application code uses the SDK programming interface for printing, instead of using the built-in interface. The SDK interface won't print if that isn't allowed by the policy.

See the [Programming Interface for Kotlin](#).

Printing prevention requires integration work by the app developer.

Integration

To integrate the feature into your application, follow the instructions below.

Compatibility

Before you begin integration of this feature, ensure you have access to compatible versions of software. The following table shows the minimum required versions.

Software	Required
Workspace ONE SDK for Android	20.11
Workspace ONE management console	9.7*

Table 1: Software Compatibility Version Numbers

Version 9.7* is the earliest supported UEM at time of writing.

Console Configuration

This feature can be configured in the Workspace ONE management console. The following instructions are intended for application developers or other users wishing to try out the feature quickly. Full documentation can be found in the online help.

1. Log in to the management console.

The dashboard will be displayed.

2. Select an organization group.

By default, the Global group is selected.

3. Navigate to: Groups & Settings, All Settings, Apps, Settings and Policies, Security Policies.

This opens the Security Policies configuration screen, on which a number of settings can be switched on and off, and configured.

4. For the Data Loss Prevention setting, select Enabled.

When Enabled is selected, further controls will be displayed.

5. For the Enable Printing setting, select No.

If you find the console user interface difficult to interpret, here's a tip. The required selection is in effect if the No next to Enable Printing has the same color scheme as the Enabled next to Data Loss Prevention.

6. Select Save to commit your changes to the configuration.

This concludes console configuration.

See also the [Appendix: Console User Interface Screen Capture](#).

Programming Interface for Kotlin

The Kotlin programming interface for Workspace ONE printing prevention is the new `WS1PrintManager` class. The methods of the `WS1PrintManager` class are similar to some of the methods in the Android `PrintManager` and `PrintHelper` classes.

Replacement Kotlin Code

The following code snippets show how to change code that uses the native classes to support printing prevention.

```
// Original PrintManager code:
val printManager = activity.getSystemService(Context.PRINT_SERVICE) as PrintManager
val printJob = printManager.print(printJobName, documentAdapter, attributes)
val printJobs:MutableList<PrintJob!> = printManager.getPrintJobs()

// Replacement for PrintManager code:
val printJob = WS1PrintManager.print(activity, printJobName, documentAdapter, attributes)
val printJobs:List<PrintJob>? = WS1PrintManager.getPrintJobs(activity)

// Original PrintHelper code:
val printHelper = PrintHelper(activity)
printHelper.printBitmap(jobName, bitmap, callback)
printHelper.printBitmap(jobName, uri, callback)

// Replacement for PrintHelper code:
WS1PrintManager.printBitmap(activity, jobName, bitmap, callback)
WS1PrintManager.printBitmap(activity, jobName, uri, callback)

// WS1PrintManager extra:
val allowed:Boolean = WS1PrintManager.isPrintingAllowed()
```

The differences are as follows.

- `PrintManager` is obtained by calling the Activity `getSystemService()` method. `WS1PrintManager` is a singleton.
- `WS1PrintManager` print methods take an additional Activity parameter.
- `PrintManger` `print()` method returns null if print job creation failed. `WS1PrintManger` `print()` method returns null if print job creation failed or if printing isn't allowed.
- `PrintManager.getPrintJobs()` never returns null. `WS1PrintManager.getPrintJobs()` returns null if printing isn't allowed.
- The `PrintHelper` constructor takes an Activity parameter. The same `WS1PrintManager` singleton is used for `PrintManager` and `PrintHelper` replacement.
- `WS1PrintManager` `PrintBitmap` methods take an additional Activity parameter.

- `WS1PrintManager.isPrintingAllowed()` returns true if printing is allowed in the DLP policy for the current end user, or false otherwise.

Note that the Android `PrintManager` and `PrintHelper` classes are final and cannot be inherited from.

In addition to the differences noted above, the `WS1PrintManager` methods have a user interface to inform the user that printing isn't allowed.

User Interface for Android

The `WS1PrintManager` `print` and `printBitmap` methods display an Android toast message if printing isn't allowed. You may want to inform the user by a different mechanism, or you may want to gray out or hide parts of your app user interface if printing isn't allowed.

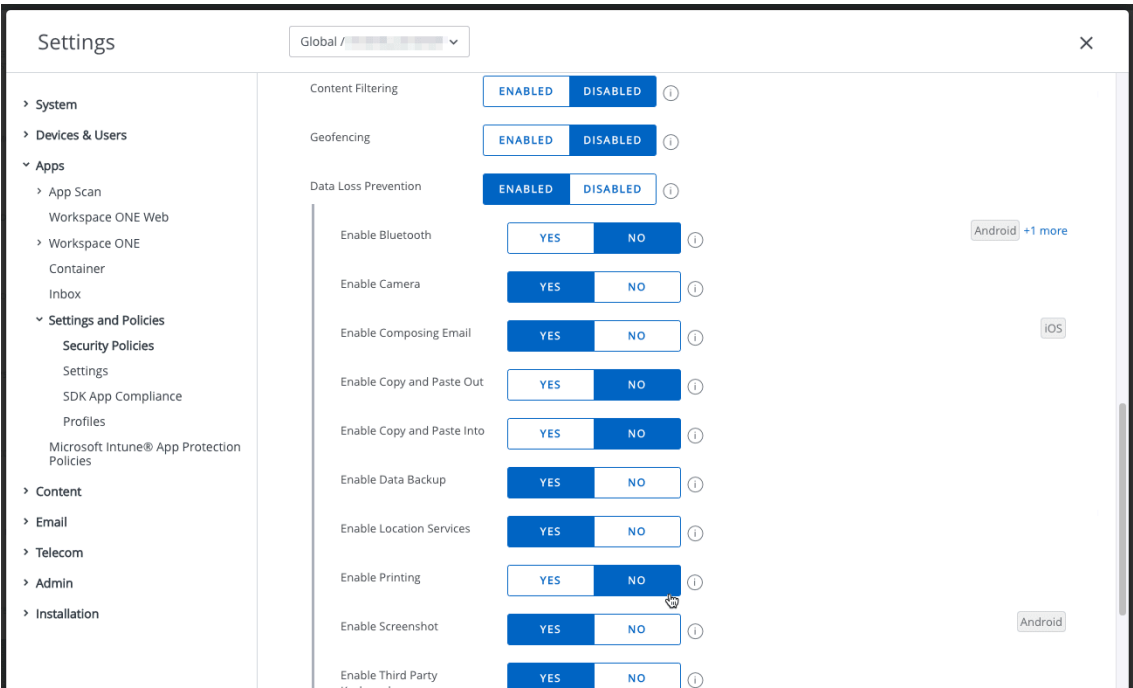
You can implement any mechanism in your own code. First call the `isPrintingAllowed()` method. Then modify your app user interface based on the return value.

Next Steps

When the required code changes have been made, the app should be tested. Install the app on a device and enroll with a UEM that has printing prevention configured in the DLP policies.

Appendix: Console User Interface Screen Capture

The following screen captures shows this feature’s configuration in the management console. For step-by-step instructions, see [Console Configuration](#).



Screen capture 1: Console User Interface Configuration

Document Information

Revision History

The following table shows the revision history of this document.

Date	Revision
12 Dec 2020	Initial Publication.
18 Feb 2025	Brand Revision.
04 Aug 2025	Updated the revision history format.

Legal

This software is licensed under the [Omnissa Software Development Kit \(SDK\) License Agreement](#); you may not use this software except in compliance with the License.

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

This software may also utilize Third-Party Open Source Software as detailed within the [Omnissa Open Source Notices](#).